Dynamic Adjustment of Monte Carlo Tree Search Simulations for a 9 × 9-Go-AI

Contents

1 Introduction

Creating a machine that is capable of playing games has been a fascination to humanity [1] and an integral part of Computer Science [2] for a long time. It is a benchmark to human intelligence and an exploration of the boundaries of what is capable using artificial decision making. This culminated in the defeat of reigning world-chess champion, Garry Kasparov, at the computations of the IBM chess computer DeepBlue in 1997 [3]. This spurred on further improvement on chess machines that is still ongoing [4] and research on a multitude of other games. One such game is the popular game by the name of Go. It remained a unbeaten challenge to scientists for 19 more years [5]. While many approaches were tested, it was not until 2016 that, on equal footing, a high-ranking professional player was beaten by Google-DeepMind's artificial intelligence (AI), AlphaGo. AlphaGo used a vastly different approach from earlier game computers and programs [6]. It combined Deep Neural Network (DNN) training with exploration guided by distributions output by the neural network. **CITE HERE**

1.1 Problem

One issue that is apparent with most AIs is the amount of ressources that is necessary for training as well as for their subsequent deployment [7]. Literature and the news are abundant with ever-growing models with needs for more and better hard-ware and energy. While improving AI's capability in all domains is an important important pursuit to stretch the limits of what is possible, it is also important to lower the costs of operating such machines. In particular, algorithms that make use of Monte Carlo Tree Search (MCTS), as in the case of AlphaGo [6] and its successors like AlphaGo Zero [8], seem to execute a lot of unnecessary computation because base MCTS executes a set number of simulations without consideration to whether all of these simulations are necessary.

1.2 Hypothesis

The central claim of this thesis is that efficiency of MCTS with predictors can be increased by dynamically adjusting the amount of simulations based on heuristics that make use probabilties generated by the predictor while upholding the same level of performance. The development and implementation of such predictive heuristics are the goal of this thesis.

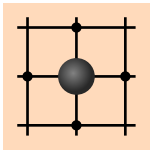

Figure 1: Liberties of stones

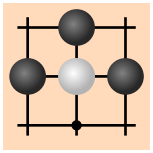

Figure 2: Capturing a Stone

2 Background knowledge

2.1 Go

Go is a Chinese board game that has a history spanning more than 4,000 years. It is played by two-players using black and white stones. While usually played on a 19 × 19-grid, other board sizes like 9 × 9 or 13 × 13 are also common. The goal of the game is to enclose more territory, meaning, grid intersections, than the opponent.

The players place stones in turn; the player with the black stones begins. The group of free places around a single stone are called liberties, see Figure 1. Stones of the same colour that are placed on neighbouring fields are considered a group and share the liberties. A group may be arbitrarily large. A group of the opponent can be captured by filling in all liberties of the group, see Figure 2. While generally any stone may be placed anywhere on the board, it may not be placed in places where the stone or its group dies because of the stone having been placed. The exception is if the stone is the cause for a successful capture of the opponent's stones. Importantly, a board position may not be immediately repeated. This means, after a stone has been captured, the capturing stone may not be recaptured on the next move. This is called Ko.

The game ends when both players pass consecutively, typically when all meaningful moves have been played and the territories are settled. Once play is concluded, the territory is counted. Unoccpied grid intersections within their territory each correspond to each point for the respective player. Captured stones each represent additional points for the player who captured them. Captured stones that remain on the board while counting are worth two points for the occupied field and the stone itself.

To compensate black's advantage in going first and avoid draw, additional 5.5-7.5 points are awarded to white. This is called komi. Komi can be adjusted to even out the differences in capability between inequally strong players. Lastly, komi can be combined with a handicap that places black stones on predetermined spots on the board.

Traditionally, players, human or computational, are attributed ranks based on strength. Beginners' ranks start at 30 kyu. The limit 1 kyu. After reaching 1 kyu, players can reach rankings from 1 dan to 9 dan. Professional players are ranked separately with their own dan-scale. Nowadays, players are also evaluated using the Elo-system.

3 Technical Background

3.1 Reinforcement Learning and Self-Play

Reinforcement Learning (RL) describes a Machine Learning (ML) paradigm in which an agent is trained to maximize a reward signal in interaction with its environment [9]. Generally during training, an agent makes use of two concepts called exploitation and exploration. On one side, to exploit is to choose a path of action that is known to produce a good result. On the other side, to explore is to choose a path of action that is not well-known and thus, upon further examination, may produce a better result than already known paths of action.

Self-play is a training scheme within the field of RL [8]. In self-play, an algorithm competes against an earlier iteration of itself. In the scenario of this paper, a training version plays a match of Go against a previous iteration of itself. In this sense, the previous iteration is part of the environment and the reward signal is the outcome of the match. Various selection strategies exist for the iteration against which is played, for example, a random iteration is chosen to play, or the selection follows a "King-of-the-Hill"-approach, where the strongest iteration remains as the benchmark and is to be beaten. This, more than the randomised selection shows growth in performance of the agent.

3.2 Deep Neural Networks

3.2.1 Artificial Neural Networks

Today, artificial neural networks (ANN) are one of the most used and researched technologies. With their groundwork laid in 1958 [10], new methods have been since been developed for a large number of purposes, not the least of which is playing games and improving beyond human capabilities.

Generally, ANNs receive an input of arbitrary format, defined per usecase. During the forward pass, the signal of this input are propagated through the network to the output layer [11]. One layer is made up of multiple neurons, each with its own weights per input,

plus a bias individual to each neuron. In the context to RL, the ANN's output is evaluated for its reward signal in response to the environment. From this, an error signal is generated and passed backwards through the network. Each layer is updated dependent on the succeeding layer's, that is in the direction of the forward pass, error signal.

3.2.2 Residual Neural Networks

In the course of research and development of ANNs, they have grown exponentially deeper. Beginning with a single perceptron, which were then used in a single layers, now, there are networks using neurons with more than 100 layers and a large number of perceptrons per layer. Users of ANNs of this size noticed the problem of the Vanishing Gradient [12]. In the process of backpropagation, the derivatives' chain rule is employed. For that, multiple small decimal numbers are repeatedly multiplied. This produces even smaller decimal numbers The Vanishing Gradient describes the phenomenon that the error signal that early layers of the networks receive is too small to effectively change the weights and biases of those layers. This stops learning from having any effect.

To solve this problem, residual neural networks (ResNet) were introduced in Deep Learning (DL) problems. ResNet's make use of "skipping connections" [12]. In one of connection, a residual layer projects its output not only to the succeeading layer but also to another layer further down the stack of layers, skipping the layers inbetween. The residual output is added to the output of the layer, to which it is is projected. This largely fixes the Vanishing Gradient since the output of the layer is passed further into the stack of layers. A negative point in this setup is that the intermediate layers may be found to be superfluous and converge to 0 during training.

3.2.3 Convolutional Neural Networks

Most objects depend on their multi-dimensional structure to be considered when analysing them. Images, for example, rely on the spatial position of pixels in relation to one another in order to correctly depict objects. Humans make use of layered groups of neurons with distinct tasks to recognise groups of features and form an understanding of objects from them.

Computationally, this structure is approximated using Convolutional Neural Networks (CNN). Similarly to their biological counterparts, CNNs find substructures that are common

for multiple objects in early layers [13]. Combining these features in later layers enables recognition of more specific types of objects. In a convolutional layer, kernels, that is, tensors, operate on a receptive field of inputs to generate outputs by sliding over the input making the operation translationally invariant instead of operating on the whole input at the same time. The weights in these layers can be learned through training.

Go, being a board game that played in two-dimensions, is especially well-suited to being analysed using CNNs. Much like recognising an object, local situations such as fights or life-and-death can be analysed in early layers and evaluated within the global context of the match in later layers. For proper recognition and evaluation of local and global situations, Go requires deep insight of the board. For more accurate representations of the global position, the combination with ResNets becomes vital to sustain trainability.

3.3 Monte Carlo Tree Search

MCTS is a versatile heuristic decision making algorithm. During the search, a tree is built based on the original state represented by root node R. Every child node i of R is a new board position, in which a different move a_i was played. Through exploration of subsequent state from R, MCTS finds a state that is most likely to maximise a metric that can be changed per use-case. Every node i of the tree keeps track of how many simulations made use of i, n_i, and how many simulations are considered wins, w_i. There are four stages to MCTS:

1. Selection: From R, choose a child node according to a selection method. Repeat until choosing a leaf node C.
2. Expansion: If C is not a terminal state, meaning, there is a possible following state, create new child nodes. Choose a new child state S.
3. Evaluation[1]: Evaluate S. S may be evaluated by choosing random subsequent states until a terminal state which has a definitive value. Another method is an evaluation function applied to S.

[1]This step is also called simulation. For the sake of clarity within the scope of this thesis, "evaluation" will be used for this step of the process.

4. Backpropagation: Update the visit and win counts in the path from S to R under consideration of the result from the evaluation.

Through rigorous exploration of the tree by simulations, meaning, paths from root to terminal nodes, the child states most likely to maximise the evaluation method are expected to converge to higher values in the selection method.

The selection method commonly used in the selection phase is the "Upper Confidence Bound 1 applied to Trees" (UCT) [14]. It is used to find a balance between exploitation of well-simulated moves and exploration of under-explored moves in MCTS. In the selection stage, node i with the highest value according to the expression

$$Q(s, a_i) + U(s, a_i) = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N_i}{n_i}} \tag{1}$$

is selected, where

- $Q(s, a) = \frac{w_i}{n_i}$ is a measure of how good a_i is in state s,
- $U(s, a) = \sqrt{\frac{\ln N_i}{n_i}}$ is a measure of how well explored move a_i is,
- w_i is the number of wins that the successor states of s accumulate if a_i is played,
- n_i is the number of times that a_i was chosen during the simulations,
- N_i is the number of times that s was part of the simulations,
- c is an exploration parameter. The higher the more explorative the move selection is.

Since $\sqrt{\frac{\ln N_i}{n_i}} \approx \infty$, every move is tried out at least once.

3.3.1 Probabilistic Upper Confidence Bound applied to Trees

The Probabilistic Upper Confidence Bound applied to Trees (PUCT) is a variant of UCT that incorporates prior estimations on the goodness of the actions of a state [8]. These evaluations are typically provided by predictors like a neural network or a domain-specific heuristics.

It uses a different uncertainty calculation:

$$U(s, a) = c_{\text{puct}} \cdot P(s, a) \cdot \frac{\sqrt{N_i}}{1 + n_i}, \tag{2}$$

where

- c_{puct} is a constant that guides the level of exploration and
- $P(s, a_i)$ is the evaluation of move a_i in state s according to the predictor.

A fundamental change is that $\sqrt{\frac{\ln N_i}{n_i}} \neq \frac{\sqrt{N_i}}{1+n_i}$ because it is accepted that not every possible move in s is tried out. Furthermore, $\ln(N_i)$ asymptotically approaches a maximum, in comparison, $\sqrt{N_i}$ keeps growing.

3.4 Kendall Rank Correlation Coefficient

The Kendall Rank Correlation Coefficient (KRCC) is a tool to calculate by how much rankings of objects of the same kind differ [15]. In the context of this thesis, objects are the predictor network's move probability distribution $\mathbf{p}$ and MCTS' node frequencey distribution $\boldsymbol{\pi}$. Positions within these distributions are always examined together. In the implementation used in this thesis, all positions are iterated through being compared to all following positions. A pair of positions is concordant if both distributions agree either that $\mathbf{p}_i < \mathbf{p}_j$ and $\boldsymbol{\pi}_i < \boldsymbol{\pi}_j$ or that $\mathbf{p}_i > \mathbf{p}_j$ and $\boldsymbol{\pi}_i > \boldsymbol{\pi}_j$. Since equal values are also possible, this thesis uses specifially τ_b. In that case, an pair is tied if $\mathbf{p}_i = \mathbf{p}_j$ or $\boldsymbol{\pi}_i = \boldsymbol{\pi}_j$. Otherwise, the pair is said to be discordant. The formula is

$$\tau_B = \frac{n_c - n_d}{\sqrt{(n_0 - n_1) \cdot (n_0 - n_2)}}, \tag{3}$$

where

- n_0 is the sum of all pairs, $n_0 = n(n-1)/2$,
- n_1 is the sum of tied values in $\mathbf{p}$, $n_1 = \sum_i t_i \frac{t_i - 1}{2}$,
- t_i is the number of tied values containing the i-th unique value,
- n_2 is the sum of tied values in $\mathbf{p}$, $n_1 = \sum_j u_j \frac{t_j - 1}{2}$, and
- u_j is the number of tied values containing the j-th unique value.

3.5 Elo

The Elo rating system is used to rank players of games such as Go and Chess. It considers the likeliness of one players win over their opponent. The central formulae are

$$E_A = \frac{1}{1 + 10^{\frac{R_B - R_A}{400}}}, \tag{4}$$

which calculates the expected result of player A with rating R_A against player B with rating R_B, and

$$R'_A = R_A + K \cdot (S_A - E_A), \tag{5}$$

which updates the ranking of player A. S_A is the actual result and can be 1 for a win, 0.5 for a draw, and 0 for a loss. K scales the difference in points. Experienced players have a

lower factor to keep their ranking more stable; newer players have a higher factor so that the rating changes more rapidly.

The result from the Elo rating system is that a player with 400 points more than his opponent is about 10 times as strong as his opponent because he should 10 games out of 11. This is because $\frac{1}{1+10} = 91\%$.

4 Previous work

4.1 Pre-deep Learning

4.1.1 Before MCTS

Early Go programs were usually rule-based like Deep Blue or used pattern recognition to find hard-coded patterns and choose actions based on those patterns.

Albert Zobrist wrote an early Go program which was based on pattern recognition and was capable of calculating territory and playing Ko's through Zobrist hashing which still has uses to this day.

Another such program is "The Many Faces of Go" that was using a large compilation of rules and an influence function. It was vastly successful in Computer Go championships in 1998-2002. It was awarded an 8-kyu rank for winning a tournament in . It has since been continuously improved and made to embrace new technologies until the introduction of Deep Learning.

An open-source program that also works based on rules and pattern recognition is GnuGo, whose pipeline is publicly available. Within the pipelin, GnuGo first analyses the board for its state, examines groups, their life-and-death, and their relation to one another. Next, it generates the moves, and lastly evaluates the moves for its effect on influence and territory. The move that has the best evaluation is played.

4.1.2 After MCTS

Previous were found to perform well in specific parts of the game like Life-and-Death-Problems or **FILL Something Else**. However they were unable to play a complete game well or even better than amateur players. Programs of this level of sophistication could not achieve the same level of success for Go as for chess, because the computational search for Go is drastically different. While it is not the only factor, the search space is much larger on a large board, $3^{19\times19} \approx 10^{170}$. However, Go on a 9 × 9-board has a similar a branching

factor to chess and has proved equally as difficult as Go on a large board. This is due static position evaluation requiring a large amount of local auxiliary computations. Additionally, in order to find legal moves, the history of the game has to be contained within the state information to account for special rules such as Ko.

As the typical game is also much longer than in chess, MCTS is a tool better suited to explorations because it is capable of a more dynamical approach. Furthermore, short-term profit may often be outpaced by an opponent's stronger tactical play. With the introduction of MCTS into the field of Computer Go, programs became drastically better. In Go on a 9 × 9-board, a 5 dan professional player was beaten by MoGo in 2007. In 2009, a 9 dan professional by Fuego. In the course of pure MCTS applied to Go on a 19 × 19-board, various models managed to defeat high-ranking professional players, albeit with up to 9 handicap stones, and retain a ranking of 4 dan on online play.

4.2 Deep Learning

4.2.1 AlphaGo

The AlphaGo-algorithm is a complex system of neural networks, MCTS, supervised learning (SL), and reinforcment learning (RL). SL policy network p_σ and SL sample network p_π were trained from a database of expert-level moves. p_π was trained to accurately predict human moves. p_π was designed to quickly sample actions during the rollout in the simulation of MCTS. RL policy network p_ρ on the other hand is trained to optimize the outcome of self-play. The RL value network $v^{p(s)}$ predicts the outcome using policy p.

Supervised learning. The first policy network$p_{\pi(a|s)}$ receives as input a board representation s and gives out movej a probability distribution over all legal moves a. It optimizes the selection of action a, given s, using stochastic gradient descent according to $\Delta\sigma \propto \frac{\delta \log p_\sigma(a|s)}{\sigma}$.

Reinforcement learning. The second policy network $p_{\pi(a|s)}$ is architecturally the same, with the same initial weights $\rho = \pi$ and trained with policy gradient reinforcement learning. It also outputs a probability distribution over all legal moves a. Self-play is done with a current and a random previous iteration of the network. The reward function is zero for all non-terminal time steps and either $+1$ or -1 for winning and losing, respetively. The loss is thus $\Delta\rho \propto \frac{\delta \log p_\rho(a_t|s_t)}{\rho} z_t$.

The value function $v^{p(s)}$ approximates the perfect play $p^*(s)$ using the best available policy p from p_ρ to $v^{p(s)} = \mathbb{E}[z_t|s_t = s, a_{t...T} \sim p]$. The value network $v_{\theta(s)}$ approximates that function. Architecturally, v_θ is similar to the policy networks but is trained on state-outcome pairs (s, z) using stochastic gradient descent to minimize the mean-squared error (MSE) between predicted value $v_{\theta(s)}$ and the actual outcome z $\Delta\theta \propto \frac{\delta v_{\theta(s)}}{\delta\theta}(z - v_\theta(s))$.

Search In the MCTS, each edge (s, a) holds record of an action value $Q(s, a)$, visit count $N(s, a)$, and prior probability $P(s, a)$. The tree is traversed by selection of a child at time step t by

$$a_t = \underset{a}{\text{argmax}}(Q(s_t, a) + u(s_t, a)) \tag{6}$$

with a bonus

$$u(s, a) \propto \frac{P(s, a)}{1 + N(s, a)} \tag{7}$$

which encourages exploration by disfavouring much-visited edges.

A leaf node L is processed once by p_π which outputs the prior probabilities $P(a|s) = p_\pi(a|s)$. $v_{\theta(s_L)}$ and $p_{\sigma(s_L)}$ **CHECK** are used to generate an evaluation $V(s_L)$ of the state of the leaf node weighted by a mixing factor λ:

$$V(s_L) = (1 - \lambda)v_{\theta(s_L)} + \lambda z_L \tag{8}$$

At the end of one iteration of search, visited edges are updated. Visits through all search iterations are counted and mean evaluation calculated to

$$N(s, a) = \sum_{i=1}^{n} 1(s, a, i)$$

$$N(s, a) = \sum_{i=1}^{n} 1(s, a, i) \tag{9}$$

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{i=1}^{n} 1(s, a, i)V(s_L^i)$$

where s_L^i is the leaf node L in the ith simulation. $1(s, a, i)$ shows whether that node was visited in the ith iteration.

4.2.2 AlphaGo Zero

AlphaGo Zero represents a significant improvement over its predecessor, achieving better performance through a more efficient architecture. By eliminating the need for human expert data, it masters the game entirely through self-play.

4.2.2.1 Structure of AlphaGo Zero

Unlike AlphaGo that uses separate policy and value networks, AlphaGo Zero makes use of a unified DNN, that consists of residual convolutional layers as well as batch normalisation and rectifier nonlinearities. This network, $f_{\theta(s)}$, takes as input a board state s and outputs both move probabilities and state evaluation:

$$(\mathbf{p}, v) = f_\theta(s). \tag{10}$$

The probability distribution $\mathbf{p}$ is a vector over all allowed moves in s, where $p_a = \Pr(a \mid s)$. The scalar value $v \in [-1, 1] \approx \mathbb{E}[z, s]$ is the expected outcome of the game, where z is the outcome of the game. If z is equal to 1, the current player wins. If z is equal to -1, the opponent wins.

The distribution $\mathbf{p}$ and scalar v guide the following MCTS in the simulations. To avoid the agent narrowing in on suboptimal solutions, Dirichlet-noise is added. A weighted Dirichlet distribution $\eta, \eta \sim \mathrm{Dir}(0.03)$, is added to $\mathbf{p}$

$$P(s, a) = \varepsilon\eta_a + (1 - \varepsilon)p_a. \tag{11}$$

During each simulation time step, the algorithm selects the action a_t that maximises PUCT. After the allotted simulations, MCTS returns a probability distribution based on visit counts:

$$\boldsymbol{\pi}(a|s_0) = \frac{N(s, a)^{\frac{1}{\tau}}}{\sum_b N(s_0, b)^{\frac{1}{\tau}}}, \tag{12}$$

where τ is a temperature variable that controls exploration. From this distribution, an action is sampled.

4.2.2.2 Training of AlphaGo Zero

AlphaGo Zero learns through an iterative self-play loop. The current best version (θ) plays against itself to generate a dataset of (s_t, π_t, z_t). The network is updated to minimise the composite loss function

$$l = (z - v)^2 + \boldsymbol{\pi} \log \mathbf{p} + c(\|\theta\|)^2. \tag{13}$$

Furthermore, the fact that Go is rotationally invariable is exploited by rotating the board and thus generating more valid game positions.

4.2.3 AlphaZero

AlphaZero is another iteration on this self-play algorithm. While ultimately the same structure as AlphaGo Zero, the domain-specific optimisation that Go is rotationally invariable is removed which reduces the amount of data of data available for this game.
AlphaZero managed to beat state-of-the-art models in chess (Stockfish), shogi (Elmo), and Go (AlphaGo Zero).

4.2.4 The Alpha-Models

There are multiple named versions of AlphaGo, each with their own achievements. AlphaGo Fan, which was the program to win against a human professional on a 9×9 board, won 5-nil. AlphaGo Lee Sedol won against Lee Sedol with a record of $4 - 1$. The last iteration, AlphaGo Master, won against top human professionals in online games with a record of $60 - 0$. When these models were compared to AlphaGo Zero, their Elo ratings were $3,144$, $3,739$, and $4,858$, respectively. AlphaGo Zero received a rating of $5,158$.

4.2.5 MuZero

MuZero is a generalisation of the Alpha-family. While previous models required a working model of the environment, MuZero is capable of learning about the environment through interaction with the environment. While also achieving similar results on traditional games such as Go and chess, it is also capable of learning to play real-time games like Atari. In all of these domains, it was capable of surpassing state-of-the-art models and therefore top human professionals.

4.2.6 Gumbel AlphaZero

Google DeepMind's successor model, Gumbel AlphaZero, breaks from traditional MCTS using UCB or variants, and instead makes use of other policy improvement strategies. One such strategy is Sequential Halving.

4.2.7 KataGo

KataGo represents a significant step to the wider Computer Go community. While stronger models such as **FILL** exist, KataGo is one the few open-source models. Improving on the architecture and process of AlphaZero, KataGo introduces and publicly explains several domain-dependent and -independent additions to improve efficiency, flexibility, and utility for human analysis.
Some of these features include:

- Auxiliary Policy Targets: In addition to simply predicting the best move for the current board position, KataGo also predicts the opponent's move.
- Playout Cap Randomisation: Most board positions are searched with a low number of simulations and only used to train the value head. Long playouts are then used for policy training.
- Blocking Overplay: Forbidding playing in a certain region after a number of moves contributes in efficiency, albeit little in regard to performance.
- Auxiliary Ownership and Score Targets: The network also predicts to whom an intersection belongs as well as a estimations of the point difference at the end of the game.

4.2.8 Further research

4.2.8.1 Dynamic MCTS

Forming a basis for this thesis, Lan et al. introduced estimations of uncertainty at intervals during training, to determine whether continued search is necessary. As MCTS finds the optimal move after 1 simulation 62% of the time, and most positions require far less than the maximal number for the same. Thus, if there is founded certainty that the best move has been found, search should terminate. **FILL more?**

5 Methodology

5.1 Reasoning 9 × 9 vs. 19 × 19

A 9×9 board has a state-space complexity of **FILL** , of which **FILL** are legal. In contrast, a 19×19 board has **FILL** possible positions and **FILL** legal positions which constitutes an increase of **FILL** percent. Thus, in order to gain comprehensive results within the hardware and temporal limits of this thesis, the board size is restricted to 9×9.

5.2 AlphaZero > Alternatives

The base model for this thesis is AlphaZero for simple reasons. Open-sourced model KataGo competes at the current state-of-the-art in Go-playing. To achieve this level, KataGo is highly optimised using a multitude of methods, including those above. Introducing a new heuristic, it is difficult to determine whether resulting performance is due to the heuristic itself or its interaction with one or more of KataGo's many existing optimisations. Furthermore, the codebase includes and is highly optimised for a variety of

use-cases. This includes training on multiple machines, the optimisations explained above, and generalisability to multiple board sizes. This combines to a complex codebase that I do not want to deal with.

Despite the successes of the Gumbel models over their predecessors, the general idea of using MCTS in such manners remains largely the pattern most models emulate.

As an open-source project, KataGo competes at the level of the current state-of-the-art in Go-playing AI. To achieve this level of performance, KataGo is highly optimised using a multitude of methods, including those mentioned above. If a new heuristic is tested on top of those optimisations, it is difficult to determine whether the resulting performance gain is due to the heuristic itself or its interaction with one of KataGo's many existing optimisations. Furthermore, KataGo's codebase is highly optimised for and includes a variety of use-cases. This includes support for distributed training on multiple machines, the optimisations explained above, and usage of multiple board sizes during the training loop. Thus, in order to increase legibility of the codebase and reducability on the implemented optimisations of this research, AlphaZero is used as a baseline. **CHECK**

FILL Using Dynamic MCTS paper as further comparison

5.3 Environment (Training server, implementation)

The primary research tool for this study is OpenSpiel, an open-source framework developed by Google DeepMind to simplify RL resarch in games. It provides a robust, community-validated implementation of the AlphaZero algorithm and native support for Go.

OpenSpiel's AlphaZero implementation and the parts are entirely written in C++ for increased performance over Python implementations. It uses OpenSpiel's custom MCTS implementation and Meta's ML-library LibTorch for high-performance. The training loop and data management are implemented in Python. Game Representation: The 9x9 Go environment in OpenSpiel follows standard Tromp-P Taylor rules. The state is represented as a 9×9×17 tensor, encoding the current board position, previous moves (for history/ko), and the current player's color.

5.3.1 Computational Resources: University HPC Cluster

Due to the high computational cost of self-play reinforcement learning, all training and evaluation experiments were conducted on the University's High-Performance Computing (HPC) Cluster.

5.3.1.1 Hardware Configuration

The cluster provides a distributed environment optimized for deep learning. The specific resources allocated for this project include:

- Compute Nodes: Training was distributed across [X] nodes, each equipped with [Y] CPU cores (e.g., AMD EPYC or Intel Xeon) to handle the parallel MCTS simulations.
- Accelerators: Neural network training and inference during self-play were accelerated using NVIDIA [Model,e.g.,A100/V100] GPUs, providing the necessary TFLOPS for high-throughput batch processing.
- Memory: Each node provided [Z] GB of RAM to maintain large MCTS search trees and experience replay buffers.

5.4 Heuristics

The heuristics suggested here are based on the idea, that the prior is able to epxress an inherent certainty that can be leveraged to adjust the amount of simulation needed to generate a successful training sequence while lowering the computational efforts.

FILL Positives and Negatives for each heuristic, why are they better? Do they actually lower computational effort?

5.4.1 Entropy of Likeliest k Actions

The goal for this heuristic is to calculate the entropy of the k highest-rated moves of the predictor's move probability distribution and adjust the amount of simulations accordingly.

Entropy is by itself a measure of uncertainty for a distribution of probabilities. The use of the entropy is therefore an apparent possible solution to the goal of this thesis. Furthermore, using the 2 likeliest options of a probability distribution is a common approach to gather a measure of certainty of that distribution.

The important formulae are

$$H_{\text{norm}} = \frac{H(\text{Top}k)}{\ln(k)}, \tag{14}$$

$$n_{\text{new}} = n_{\text{base}} \cdot \frac{2}{1 + e^{-s(H_{\text{norm}} - sp)}} \tag{15}$$

where

- $H(\text{Top}k)$ is the entropy of the k highest ranked moves of π that are sorted descendingly in $\text{Top}k$.
- H_{norm} is the result of re-normalising the entropy of these moves to within the range $[0, 1]$,
- n_{new} is the applied number of simulations,
- n_{base} is the baseline number of simulations,
- s is the steepness of the graph, and
- sp is the position of the inflection point on the x-axis.

Equation 15, represented in Figure 3, is a logistic function. The formula here will continue to be used with $s = 8$ and $sp = 0.5$. The graph is maximised at $\frac{2}{1+e^{-8(1-0.5)}} \approx 1.964$ and, because H is normalised, it is minimised at $\frac{2}{1+e^{-8(0-0.5)}} \approx 0.036$. This should allow the entropy of the k highest-rated to transition fluidly around the inflection point at 0.5.

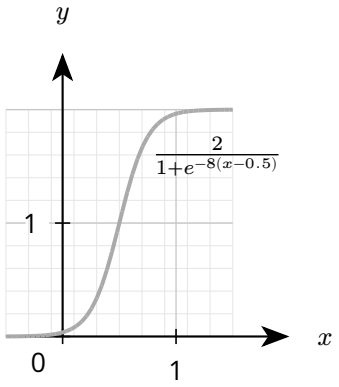

Figure 3: Logistical uncertainty function

Effort for this heuristic should be near negligible. The algorithmic complexity of finding the highest-sorted $\text{O}(|A| \cdot \log k)$, with $|A|$ as the space of all actions. There also is the additional cost of calculating the entropy of $\text{Top}k$.

Making use of more values of that distribution should also provide a clearer estimation of certainty, differences are discussed in a later section.

5.4.2 Comparison of High And Low Ranking Moves

This heuristic measures the disparity between the probability densities of the k highest-rated moves and the $A - k$ lowest-rated moves. Using this distance, the number of simulations is adjusted.

The above heuristic has a strong focus on relatively few high-rated moves compared to the overall space of possible moves. This increases vulnerablity to overconfidence where it is not warranted. That is especially the case when a single over is disproportionately well-rated. Therefore, one solution to this problem can be a more global consideration of the move probability space.

$$d = \sum_{a \in \text{ Top}k} \pi(a) - \sum_{a \in A \setminus \text{ Top}k} \pi(a), \tag{16}$$

$$n_{\text{new}} = n_{\text{base}} \cdot (\frac{2}{}, \tag{17}$$

where

- $\pi(a)$ is the probability of a in π,
- $\text{Top}k$ is the array of the highest-rated k actions,
- A is the space of all possible actions.

Equation 17 is the same as Equation 15 with the entropy exchanged for the difference of the probability densities.

This solution should be able to help mitigate the inequal consideration of moves from the first heuristic. Computational effort consists of the sorting in $O(n \log k)$ and the addition. Therefore the effort should be about as much as for the first heuristic.

Differences in k here will also be discussed in a later section.

5.4.3 Auxiliary Head Estimating Uncertainty

Lan et al. introduced supplementary networks to estimate certainty and stop searching when reaching a certain threshold. However, as merging networks for predicting value and policy has proved successful previously, enabling the network to predict its own prediction-search disagreement should also turn out beneficial. The predictor network f on board position s will return

$$f(s) = (\mathbf{p}, v, u), \tag{18}$$

where $\mathbf{p}$ is the move probability distribution, v the evaluation, and u the uncertainty estimation. The number of simulations is adjusted according to the formula

$$n_{\text{new}} = n_{\text{baseline}} \cdot \exp(u). \tag{19}$$

This means, that searching more thoroughly correlates with higher uncertainty while a lower number of simulations correlates with less uncertainty. The exponential function

allows both to increase and decrease the number of simulations. n_{new} also is limited to a maximal number to stop the search from going unendingly long.

The loss will be calculated using the expression

$$l_{\text{all}} = l_{\text{az}} + c \cdot (u - \lambda \cdot \tau(\mathbf{p}, \pi))^2, \tag{20}$$

where

- l_{az} is the error signal used by AlphaZero considering $(\mathbf{p}, \sigma, v, \theta)$),
- c is a weighting constant for MSE,
- $\tau(\mathbf{p}, \pi)$ is the KRCC of board state s,
- λ is a scaling constant for the KRCC, and
- π is the distribution found by MCTS.

In this context $\tau(\mathbf{p}, \pi)$ measures the similarity between the ranked move distribution estimated by the prior and the ranked move distribution determined by MCTS.

This heuristic should be capable of reducing the average simulation numbers by predicting how much move estimations change during MCTS in different positions. However, especially the process can be very time-intensive in the beginning of a single game when more moves are available and the calculation of KRCC requires more effort.

While the idea is promising, a glaring problem with this approach is the computational cost of calculating the KRCC for the two distributions. One solution is to similarly to the above heuristics consider only the highest k actions. This is also looked at in the following sections.

5.4.4 Random

Following the example of Wu et al., a uniformly chosen number from $n \in [1, 300]$ will be used as the number of playouts for MCTS in order to investigate in comparison whether these strategies actually fulfill their purpose and have an active part in lowering computational effort. If this performs equally well as the strategies suggested above and assuming that they all perform similarly well to the base line model, this would suggest that these strategies constitute unnecessary computational effort.

5.5 Evaluation

After the models are trained, they play against each other **FILL certain amount of games** to evaluate playing strength. To further evaluate ability, the BayesElo-algorithm will be

used. **FILL or α-Rank** The models also record the number of simulations executed from MCTS during training as well as test-play to enable comparisons between computational effort spent. As a method of comparing the efficiency of models, the following formula may give insights into the differences:

$$SE = \frac{\Delta \text{ Elo}}{\Delta \text{ Simulations}} \tag{21}$$

6 Experiments

6.1 Tuning Hyperparameters

For each heuristic, the size of the window of consideration may play a significant role in the performance of the model and effort of the heuristic. Therefore, to find a balance between performance and effort, $k \in \{2, 5, 10\}$ are trained for each heuristic and compared to performance.

Similarly, limiting the window of consideration to the most likely moves for the last heuristic may also

For the first three heuristics, k may play a huge role in accuracy and effort. Therefore, to find a good k, these approaches are first run with $a \in \{2, 5, 10\}$. The models will be given 12 hours to train and are conclusively compared. The hyperparameter with the best performance will be continued to be used throughout the other experiments.

6.2 Simple Usage

6.3 Forced Maximal Number Simulations

Wu et al. noticed that using fewer simulations is permissible as long as there are correcting playouts of sufficient length to correct the gradient of learning. However, KataGo unilaterally cuts of exploration of a board position after the lowered number of simulations during most board searches. The heuristics proposed here aim at precisely that point at which further exploration of the state becomes less useful than exploring the next state.

6.4 Pruning Superfluous Child Nodes

Previous results show improved performance by pruning unnecessary nodes so that nodes of a higher probability can be more rigorously tested. This might prove especially useful in the use cases of these heuristics that put a lower limit on available simulations.

6.5 Warm-up

7 Discussion

8 Conclusion

Bibliography

[1] K. G. von Windisch, *Inanimate Reason; or, a Circumstantial Account of That Astonishing Piece of Mechanism, M. de Kempelen's Chess-Player*. S. Bladon, 1784.

[2] A. M. Turing, "Digital Computers Applied to Games," in *Faster Than Thought: A Symposium on Digital Computing Machines*, B. V. Bowden, Ed., Pitman, 1953.

[3] IBM, "Deep Blue." [Online]. Available: https://www.ibm.com/history/deep-blue

[4] IBM, "Stockfish Blog." [Online]. Available: https://stockfishchess.org/blog/2026/stockfish-18/

[5] DeepMind, "AlphaGo." [Online]. Available: https://deepmind.google/research/alphago/

[6] D. Silver *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, Jan. 2016, doi: 10.1038/nature16961.

[7] AI Index Steering Committee, "Artificial Intelligence Index Report 2025," Stanford Institute for Human-Centered AI (HAI), Apr. 2025. [Online]. Available: https://hai.stanford.edu/ai-index/2025-ai-index-report

[8] D. Silver *et al.*, "Mastering the game of Go without human knowledge," Oct. 19, 2017. doi: 10.1038/nature24270.

[9] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[10] F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, no. 5, Nov. 1958, doi: 10.1037/h0042519.

[11] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, Oct. 1986, doi: 10.1038/323533a0.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition*, June 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

[13] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition,," in *Proceedings of the IEEE*, Nov. 1998, pp. 2278–2324. doi: 10.1109/5.726791.

[14] L. Kocsis and C. Szepesvári, “Bandit Based Monte-Carlo Planning,” 2006. doi: 10.1007/11871842_29.

[15] M. G. Kendall and J. D. Gibbons, “Tied Ranks,” in *Rank Correlation Methods*, 5th ed., 1990, pp. 40–60.